

Rahul Koju

DevOps Engineer

9840456322 | Bhaktapur, Nepal | contact@rahulkoju.com.np | [Portfolio](#) | [GitHub](#) | [LinkedIn](#)

PROFESSIONAL SUMMARY

DevOps engineer with hands-on experience building containerised applications, CI/CD pipelines, and cloud infrastructure on AWS. Designed and operated production-grade Kubernetes clusters with automated GitOps deployments, failure recovery, and full Prometheus/Grafana/Loki observability. Authored Terraform and Ansible IaC to provision repeatable AWS environments and implemented defence-in-depth networking with IAM least-privilege access. Seeking to contribute infrastructure automation and observability skills to a DevOps team.

SKILLS & EXPERTISE

CI/CD & GitOps: GitHub Actions, Jenkins, ArgoCD, GitOps, Bash scripting, Python scripting

Containers & Orchestration: Docker, Docker Compose, Kubernetes, RKE, Helm

Cloud & IaC: AWS (EC2, VPC, IAM, S3, Security Groups), Terraform, Ansible

Observability: Prometheus, Grafana, Loki, Alertmanager, health checks

Networking & Security: Linux administration, TLS/SSL, TCP/IP, DNS, HTTP/S, Nginx, cert-manager, reverse proxy

Backend: Node.js, REST APIs, Postgres, MySQL, MongoDB

Frontend: React, Next.js, Vite

Languages & Tools: Go, TypeScript, Bash, Python, Git, Linux

WORK EXPERIENCE

Frontend Developer

Feb 2025 - May 2026

Lipi Point Pvt. Ltd., Bhaktapur, Nepal

- Cut container image sizes by 92% (3–4 GB to 200–350 MB) and reduced EC2 cold-start time by ~15s by authoring multi-stage Dockerfiles with Turborepo pruning across a 3-service monorepo, enabling faster, lighter production deployments.
 - Eliminated environment-drift incidents across dev and production by configuring Docker Compose with isolated bridge networking, restart policies, and environment-variable injection across 3 services, achieving consistent, repeatable deployments.
 - Reduced production misconfiguration incidents by ~80% by redesigning REST API config to environment-variable separation with structured error handling, replacing a single-config setup that caused repeated deployment failures.
 - Cut mean time to resolve (MTTR) production incidents by ~70% by standardizing Docker health checks and structured JSON logging across all 3 Next.js services, enabling faster log analysis and reducing average debug-to-fix cycles from hours to minutes.
 - Automated image optimisation, gallery generation, and asset pipeline tasks using Python and Bash scripts, eliminating ~2 hours of manual work per release and reducing per-deploy asset processing time by ~40%.
 - Authored deployment runbooks, onboarding SOPs, infrastructure setup guides, and system architecture docs across 2 production projects, reducing new team member ramp-up time by ~50% and establishing a reliable incident response reference used across the team.
-

PROJECTS

Commit – Three-Tier Web Application [Github](#) | [Live](#)

Jan 2026 - Present

Personal productivity app (tasks, habits, notes, focus timer) built with React, Go, and Postgres, deployed on Kubernetes with GitOps, full observability, and automated CI/CD on AWS.

- Provisioned the full AWS environment (VPC, networking, security groups, IAM resources) with Terraform in ~50 seconds, cutting infrastructure setup from a multi-hour manual process to a single repeatable script.
- Automated Linux node configuration (swap, kernel modules, sysctl tuning) with Ansible in ~65s and bootstrapped a 2-node RKE Kubernetes cluster with Canal CNI and NGINX Ingress in ~2m 25s, cutting total cluster setup from a multi-

hour manual process to under 10 minutes from a blank AWS environment to a fully operational, monitored production stack.

- Eliminated configuration drift across all environments by authoring Kubernetes manifests for 3 application tiers with ConfigMaps, Secrets, and a Postgres StatefulSet backed by persistent storage, replacing ad-hoc kubectl changes with fully declarative, version-controlled configuration.
- Built a GitHub Actions CI/CD pipeline that builds Docker images tagged with commit SHA and auto-updates deployment manifests, reducing release effort to zero with consistent sub-3-minute deployments.
- Implemented GitOps delivery with ArgoCD, auto-syncing all 3-tier manifests on every Git commit, providing declarative deployments with a full audit trail and zero manual kubectl interventions in production.
- Built a full observability stack (Prometheus, Grafana, Loki/ELK-equivalent, and Alertmanager) covering cluster metrics, pod logs, and application traces, with API latency dashboards and automated email alerting that reduced mean time to detect (MTTD) anomalies across all three application tiers.
- Documented the complete system architecture using Mermaid diagrams covering the AWS infrastructure, Kubernetes cluster topology, CI/CD pipeline, GitOps workflow, and observability stack, making the project easy to hand off and maintain.
- Hardened cluster networking with restrictive AWS Security Groups, allowlisting only required ports (etcd 2379-2380, kubelet 10250, Canal VXLAN 8472 UDP) and enforcing least-privilege IAM and VPC-internal access throughout.

Two-Tier Web Application [Github](#) | [Live](#)

- Engineered a 4-stage Jenkins CI/CD pipeline (build, test, deploy, verify) for a containerized Next.js/Postgres application on AWS EC2, achieving consistent sub-2-minute-30-second deployments with a fully automated, zero-touch release track.
- Implemented zero-touch automated rollback using versioned Docker image tags and a last-known-good marker file, enabling health-check-triggered recovery with no manual intervention required and no failed deployments reaching production.
- Conducted structured incident response drills to validate production resilience across three failure scenarios: ~5s crash recovery (SIGTERM), ~20s database outage recovery with graceful 503 degradation, and ~116s full EC2 reboot recovery, all without manual intervention; documented findings through RCA reports and operational runbooks.
- Secured Postgres credentials via Jenkins credential binding, keeping secrets out of source control and enforcing least-privilege access across all deployment stages.

Tech Stack: Next.js, Postgres, Docker Compose, Jenkins, Nginx, AWS EC2, Prisma, Certbot SSL

MeroCV [Live](#)

- Reduced production Docker image size by ~90% across a 2-app Next.js monorepo by authoring multi-stage Dockerfiles with Turborepo pruning, copying only the standalone Next.js bundle into the production runner stage.
- Eliminated race-condition failures across 2 interdependent containerized services by configuring Docker Compose with healthcheck-gated startup ordering, isolated bridge networking, and environment-variable injection, achieving stable, reproducible deployments across all environments.
- Cut per-deploy asset pipeline time by ~40% by authoring Python and Bash automation scripts for image optimization and asset processing, removing manual intervention from the build-and-release workflow.
- Architected role-based access control (RBAC) and a reusable component system for a production resume builder, while maintaining architecture documentation, deployment runbooks, and component SOPs, reducing maintenance overhead and accelerating contributor onboarding.

EDUCATION

Bachelor of Science in Computer Science and Information Technology (BSc. CSIT)

Bhaktapur Multiple Campus, Bhaktapur, Nepal

2022 - 2026